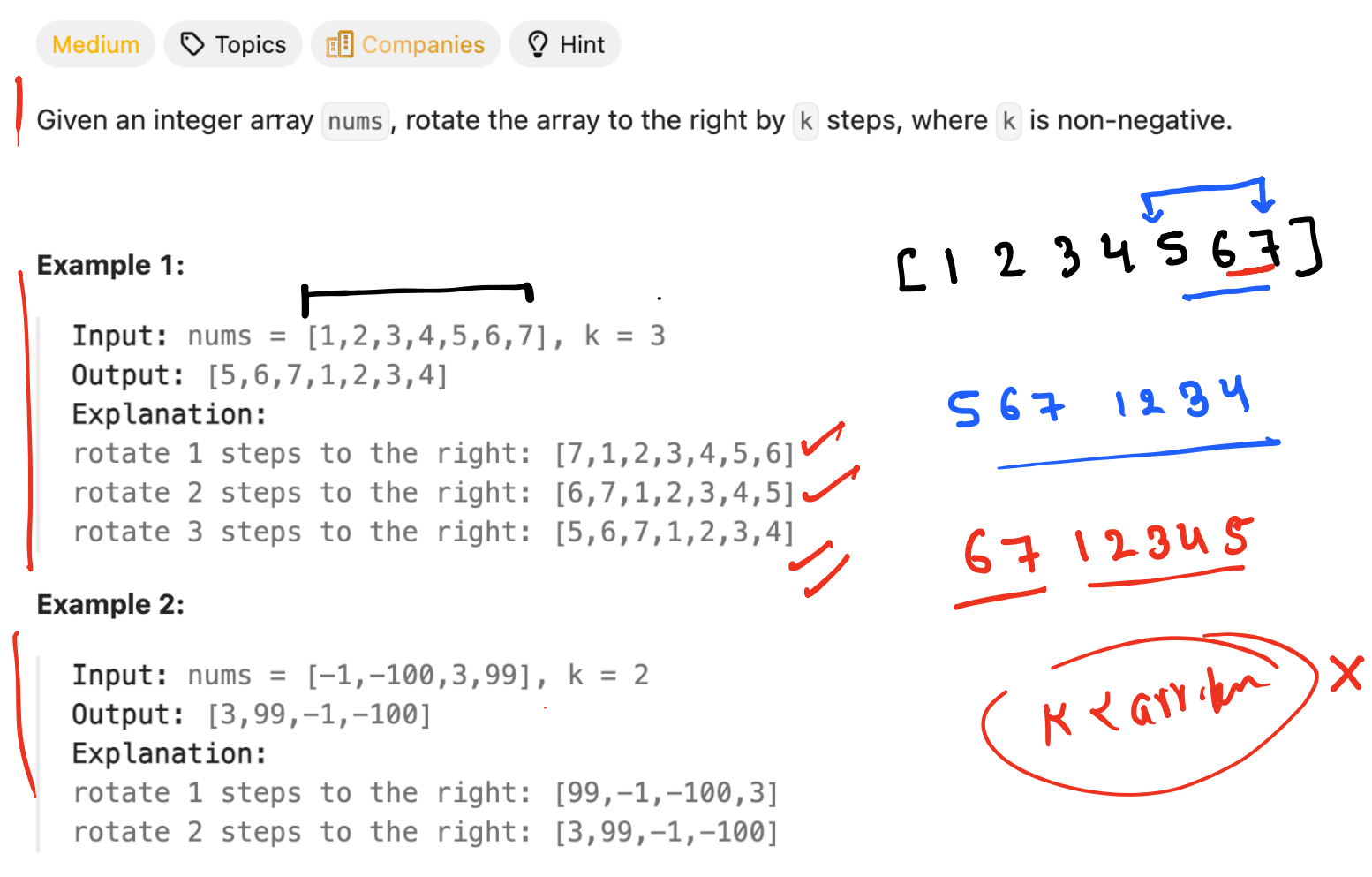
Lec-9

15/05/26 7:10 PM



Medium Topics Companies Hint

Given an integer array nums, rotate the array to the right by k steps, where k is non-negative.

Example 1:

```
Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation:
rotate 1 steps to the right: [7,1,2,3,4,5,6]
rotate 2 steps to the right: [6,7,1,2,3,4,5]
rotate 3 steps to the right: [5,6,7,1,2,3,4]
```

Example 2:

```
Input: nums = [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]
Explanation:
rotate 1 steps to the right: [99,-1,-100,3]
rotate 2 steps to the right: [3,99,-1,-100]
```

[1 2 3 4 5 6 7] K=3

K=2

5 6 7 1 2 3 4

6 7 1 2 3 4 5

K < arr.len ×

K=142

142

K = arr.length

142%7=2

N=24 15%4

5 10 15 20 25

21 22 23 24

1 2 3 4

	1 2 3 4 5 6 7
K=1	7 1 2 3 4 5 6
K=2	6 7 1 2 3 4 5
K=3	5 6 7 1 2 3 4
K=4	4 5 6 7 1 2 3
K=5	3 4 5 6 7 1 2
K=6	2 3 4 5 6 7 1
K=7	1 2 3 4 5 6 7
K=8	7 1 2 3 4 5 6
K=9	6 7 1 2 3 4 5
K=10	5 6 7 1 2 3 4

K > N

K = K % arr.length

142%7=2

7) 149 (20

140

2

3=?

3%7=3

K = K % N → length

142

1 2 3 4 5 6 7 K=3

7 1 2 3 4 5 6

6 7 1 2 3 4 5

5 6 7 1 2 3 4

N = arr.length

K = K % N

1 2 3 4 5 6 7 → 5 6 7 1 2 3 4 K=3

7 1 2 3 4 5 6

7 1 2 3 4 5 6

3%7=3

for(i=n-2 i>=0 i--){

arr[i+1]=arr[i]

① item = arr[N-1]

7 1 2 3 4 5 6

0 1 2 3 4 5 6

⑪

i=5 arr[6]=arr[5]

i=4 arr[5]=arr[4]

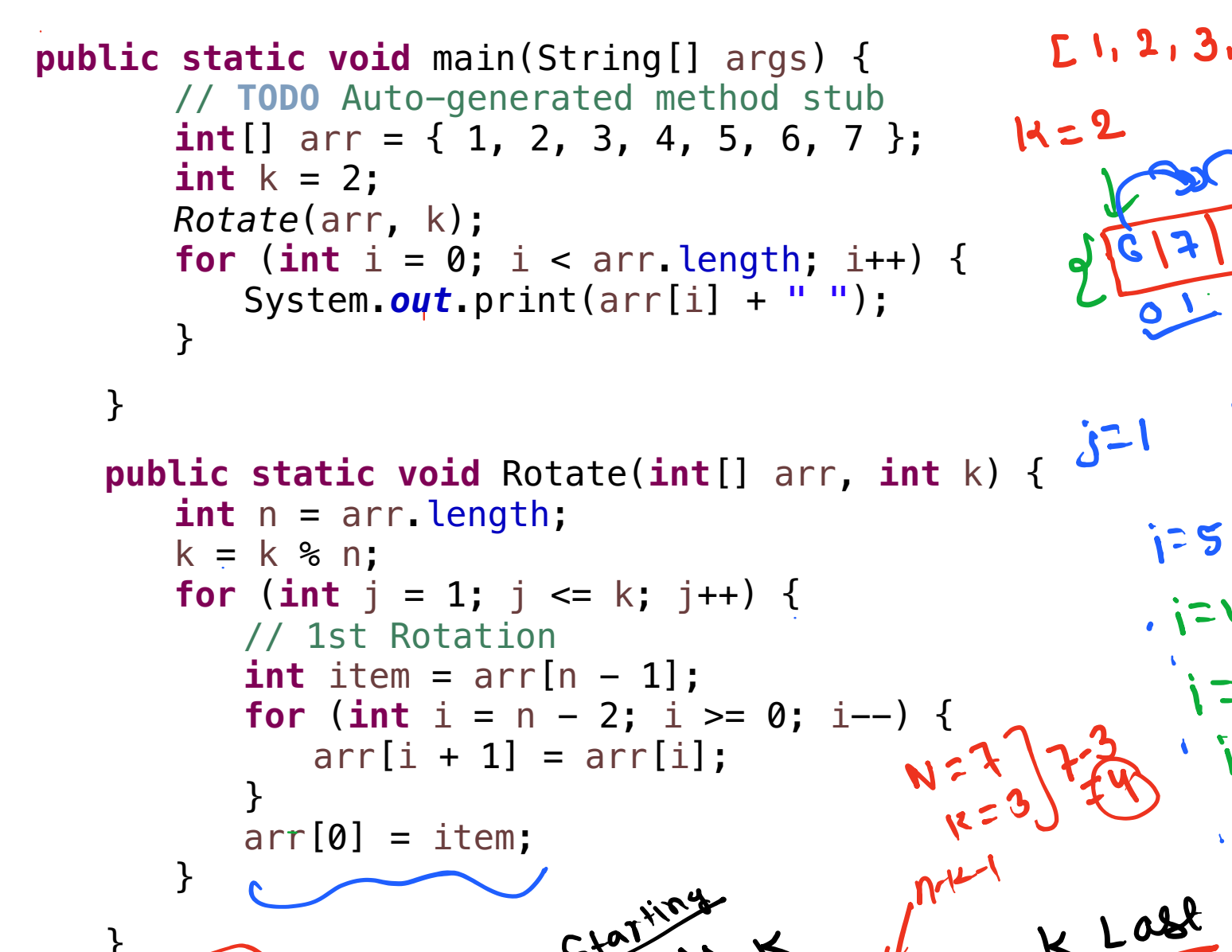
i=3 arr[4]=arr[3]

i=2 arr[3]=arr[2]

i=1 arr[2]=arr[1]

i=0 arr[1]=arr[0]

⑪ arr[0]=item



```
public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 1, 2, 3, 4, 5, 6, 7 };
        int k = 2;
        Rotate(arr, k);
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }

    }

    public static void Rotate(int[] arr, int k) {
        int n = arr.length;
        k = k % n;
        for (int j = 1; j <= k; j++) {
            // 1st Rotation
            int item = arr[n - 1];
            for (int i = n - 2; i >= 0; i--) {
                arr[i + 1] = arr[i];
            }
            arr[0] = item;
        }

    }
```

[1, 2, 3, 4, 5, 6, 7]

K=2

6 7 1 2 3 4 5

0 1 2 3 4 5 6

j=1 item=6

i=5 arr[6]=arr[5]

i=4 arr[5]=arr[4]

i=3 arr[4]=arr[3]

i=2 arr[3]=arr[2]

i=1 arr[2]=arr[1]

i=0 arr[1]=arr[0]

N=7 K=3 7-3=4

n-k-1

Starting N-K

K Last

Reverse Rev

i=0 j=n-k-1

0 1 2 3

1	2	3	4	5	6	7
4	3	2	1	7	6	5

K=3

1 2 3 4

5 6 7 1 2 3 4

5 6 7 4 3 2 1

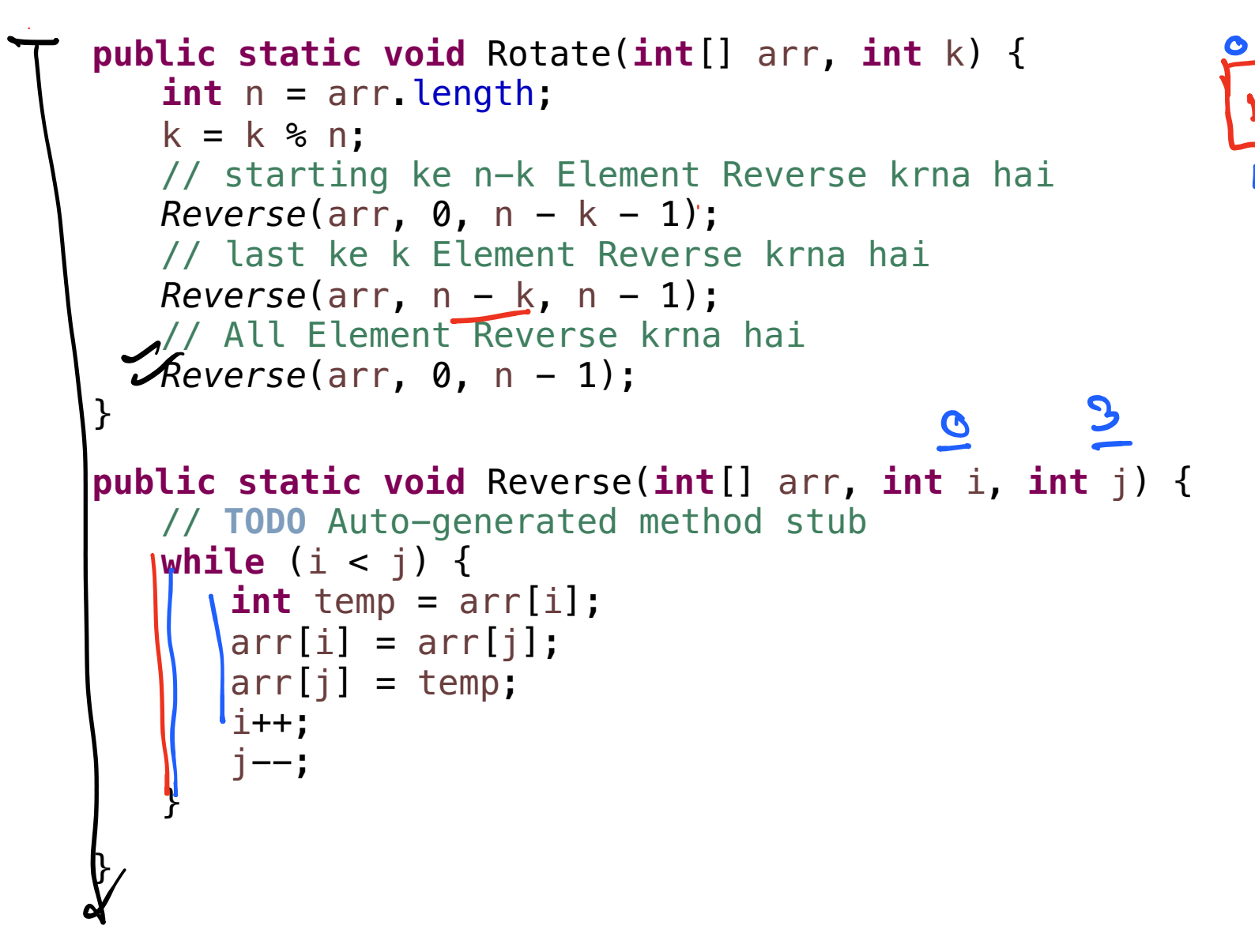
1 2 3 4

5 6 7 1 2 3 4

① Last → K element

1st → n-K element

⑪ All rev.



```
public static void Rotate(int[] arr, int k) {
    int n = arr.length;
    k = k % n;
    // starting ke n-k Element Reverse krna hai
    Reverse(arr, 0, n - k - 1);
    // last ke k Element Reverse krna hai
    Reverse(arr, n - k, n - 1);
    // All Element Reverse krna hai
    Reverse(arr, 0, n - 1);
}
public static void Reverse(int[] arr, int i, int j) {
    // TODO Auto-generated method stub
    while (i < j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
        i++;
        j--;
    }
```

0 3

0 1 2 3 4 5 6

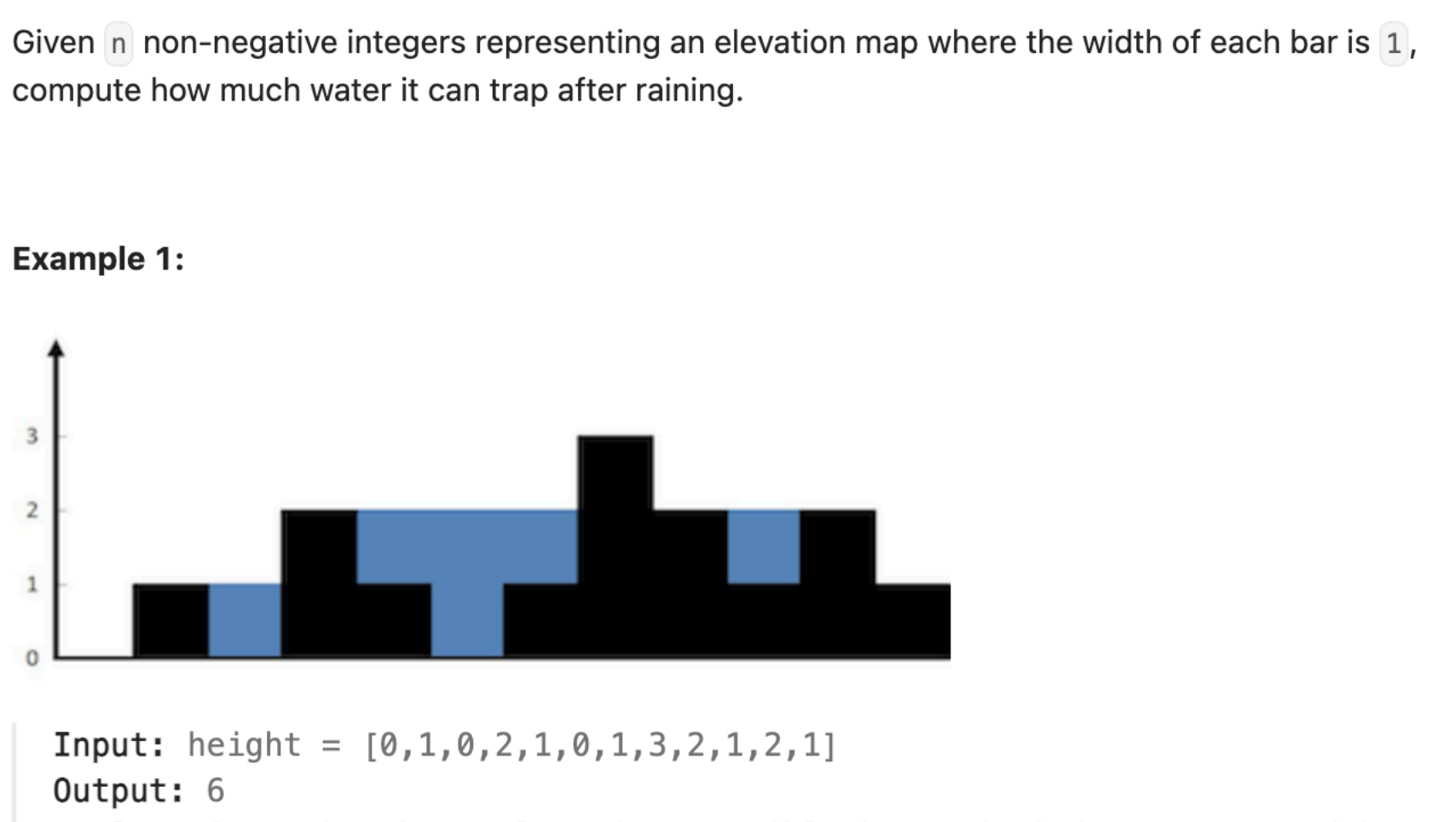
1	2	3	4	5	6	7

4 3 2 1

4	3	2	1	5	6	7

4	3	2	1	7	6	5

5 6 7 1 2 3 4



Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1:

```
Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6
Explanation: The above elevation map (black section) is represented by
array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue
section) are being trapped.
```

16

0 1 2 3 4 5 6 7

[5, 3, 1, 2, 7, 4, 1, 6]

2×1=2

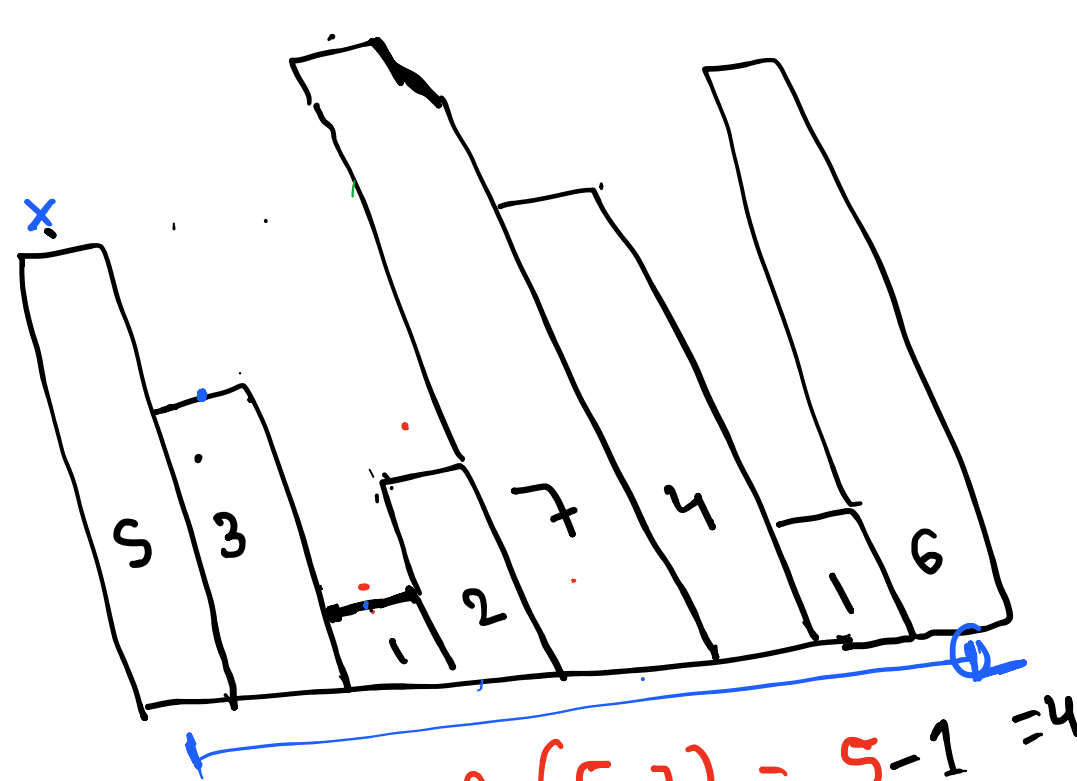
4×1=4

3×1=3

2×1=2

5×1=5

16



(0 — i) → max

(i — n-1) → max

5	5	5	5	7	7	7	7
0	1	2	3	4	5	6	7

Left

7	7	7	7	7	6	6	6
0	1	2	3	4	5	6	7

min(5,7) = 5-1 = 4×1 = 4

i=0 min(5,7) = 5-5 = 0

i=1 min(5,7) = 5-3 = 2×1 = 2

i=2 min(5,7) = 5-1 = 4×1 = 4

i=3 min(5,7) = 5-2 = 3×1 = 3

i=4 min(7,7) = 7-7 = 0

i=5 min(7,6) = 6-4 = 2×1 = 2

i=6 min(7,6) = 6-1 = 5×1 = 5

i=7 min(7,6) = 6-6 = 0×1 = 0

16

For(i=0 i<n i++){

Sum = Sum+(min(Left[i], Right[i])-arr[i])

}

0 1 2 3 4 5 6 7

[5, 3, 1, 2, 7, 4, 1, 6]

Left[0]=arr[0]

5	5	5	5	7	7	7	7
0	1	2	3	4	5	6	7

For(i=1 i<n i++){

Left[i]=max(Left[i-1],arr[i])

}

i=1 Left[1]=max(Left[0],arr[1])

i=2 Left[2]=max(Left[1],arr[2])

i=3 Left[3]=max(Left[2],arr[3])

i=4 Left[4]=max(Left[3],arr[4])

i=5 Left[5]=max(Left[4],arr[5])

i=6 Left[6]=max(Left[5],arr[6])

i=7 Left[7]=max(Left[6],arr[7])

0 1 2 3 4 5 6 7

[5, 3, 1, 2, 7, 4, 1, 6]

N=8

n-1=7

R[N-1]=arr[N-1]

5	7	7	7	7	6	6	6
0	1	2	3	4	5	6	7

i=6 R[6]=max(R[7], arr[6])

i=5 R[5]=max(R[6],arr[5])

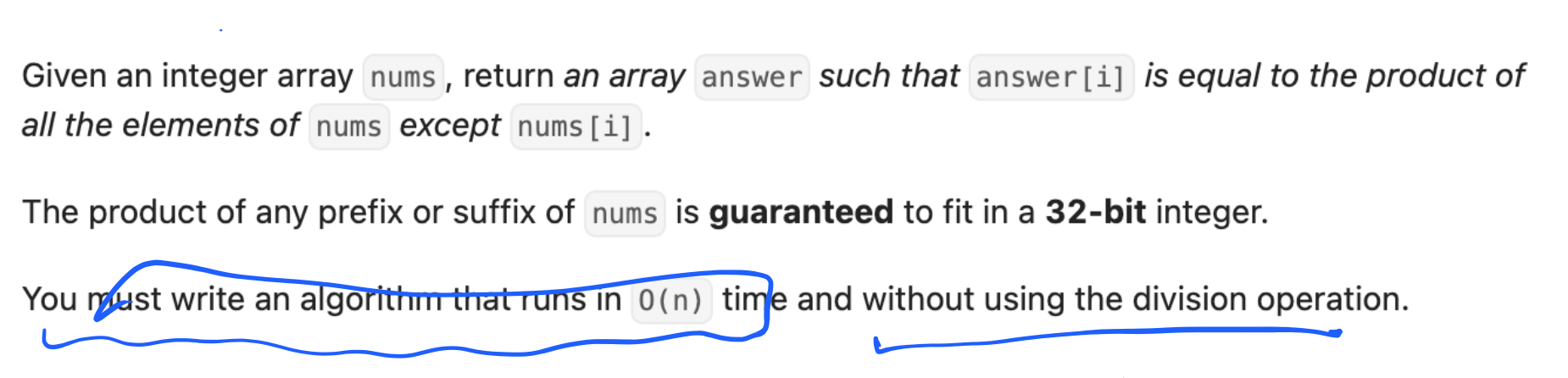
i=4 R[4]=max(R[5],arr[4])

i=3 R[3]=max(R[4], arr[3])

i=2 R[2]=max(R[3],arr[2])

i=1 R[1]=max(R[2],arr[1])

i=0 R[0]=max(R[1], arr[0])



Given an integer array nums, return *an array* answer *such that* answer[i] *is equal to the product of all the elements of* nums *except* nums[i].

The product of any prefix or suffix of nums is **guaranteed** to fit in a **32-bit** integer.

You must write an algorithm that runs in O(n) time and without using the division operation.

pre

2	3	4	5
0	1	2	3

= 120

60	40	30	24

1	2	6	24
0	1	2	3

i=1 Left[1]=Left[0]×arr[0]

i=2 Left[2]=Left[1]×arr[1]

i=3 Left[3]=Left[2]×arr[2]

Left[i]=Left[i-1]×arr[i-1]

2 × 2

1	2	6	24

60	20	5	1

3×5

4×5

60	40	30	24
0	1	2	3

60	20	5	1
0	1	2	3

R[i]=R[i+1]×arr[i+1]

R[2] = R[3]×arr[3]

R[1] = R[2]×arr[2]

R[0] = R[1]×arr[1]